

Lab - 2:

Experiment No: 2

Dated:

AIM: Create a Blockchain using Python

Lab Objectives: To implement public and private Blockchain.

Lab Outcomes (LO): Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed :

1. Make a copy of this [Google Colab Notebook](#)
2. Try to solve the errors in given Program
3. After successful execution of the Program in Colab Notebook.
 - **Add a method, create_Transactions**
 - Mine the block only when the transaction list is not null.
 - Remove the transactions from the list of transactions before mining.
 - **Modify the method, proof_of_work() to search for the golden nonce**
 - Cryptographic Puzzle is to have “000” leading zeros in the Block Hash
4. Download the code - [blockchain.py](#)
5. **Update the code to incorporate the changes in step 3 to the code in step 4.**
6. Follow the steps in [Manual](#) to demonstrate the working of Blockchain using Flask and Postman.

Tools & Libraries used :

- [Flask](#) : pip install Flask
- Download **Postman** from <https://www.postman.com/>
- Python Libraries : **datetime, jsonify, hashlib**

Instructions : (Prepare for viva for the following topics)

1. What is a Blockchain?
2. Process of Mining
3. How to check the validity of blocks in a Blockchain

Outcome :

1. Understood the concept of mining in Blockchain.
2. Implemented programs
 - Mine a Block
 - Display the chain in the Blockchain
 - Check whether the chain is valid
3. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
4. Submit the hardcopy **by the 1st week of Sem**
(As per the instructions, submit a hard copy of the same).

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes
- **json** → to encode blocks as JSON for hashing
- **Flask** → to run the blockchain API server

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
 - store the chain
 - implement Proof of Work
 - validate chain integrity
-

3. Flask Web API

Provides endpoints:

- `/mine_block` → mine a new block
 - `/get_chain` → fetch full blockchain
 - `/is_valid` → verify blockchain
-

COMPLETE CODE WITH COMMENTS (COPY-PASTE READY)

```
# -----  
# Module 1 - Create a Simple Blockchain (Fully Commented)  
# -----  
  
# Required installation:(run cmd as administrator)  
# pip install flask==2.2.5  
  
# Importing libraries  
import datetime      # To generate timestamps for blocks
```

```

import hashlib      # To generate SHA256 hashes
import json        # To convert block data into JSON
from flask import Flask, jsonify    # To create API endpoints

# -----
# Part 1 - Building the Blockchain
# -----

class Blockchain:

    def __init__(self):
        # This list will store the entire chain of blocks
        self.chain = []

        # Create the genesis block (first block)
        # proof = 1 → arbitrary
        # previous_hash = '0' → since no previous block exists
        self.create_block(proof=1, previous_hash='0')

    def create_block(self, proof, previous_hash):
        """
        Creates a new block and adds it to the chain.
        Each block contains:
        - index
        - timestamp
        - proof (PoW output)
        - previous block hash
        """
        block = {
            'index': len(self.chain) + 1,          # Position of
block in chain
            'timestamp': str(datetime.datetime.now()), # Current
timestamp
            'proof': proof,                        # PoW result

```

```

        'previous_hash': previous_hash                # Hash of the
previous block
    }

    # Add block to the chain
    self.chain.append(block)
    return block

def get_previous_block(self):
    """
    Returns the last block in the chain.
    """
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    """
    Simple Proof of Work (PoW) algorithm:
    - Find a number (new_proof)
        - Such that the SHA256 hash of (new_proof^2 -
previous_proof^2)
        starts with '0000'

    This is a computational puzzle to secure the network.
    """
    new_proof = 1
    check_proof = False

    while check_proof is False:
        # Cryptographic puzzle: hash difference of squares
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()

        # Check if hash begins with 4 leading zeros

```

```

        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1

    return new_proof

def hash(self, block):
    """
    Creates a SHA256 hash of a block.
    - Sort keys to ensure consistent hashing
    """
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    """
    Validates the blockchain by checking:
    1. The previous_hash field matches the actual hash of the
previous block.
    2. The PoW condition (hash starting with '0000') is satisfied
for each block.
    """
    previous_block = chain[0]    # Genesis block
    block_index = 1              # Start checking from block 2

    while block_index < len(chain):
        block = chain[block_index]

        # Check previous hash correctness
        if block['previous_hash'] != self.hash(previous_block):
            return False

        # Validate Proof of Work

```

```

        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()
        ).hexdigest()

        if hash_operation[:4] != '0000':
            return False

        # Move to next block
        previous_block = block
        block_index += 1

    return True

# -----
# Part 2 - Mining our Blockchain (Flask API)
# -----

# Initialize Flask web application
app = Flask(__name__)

# Create an instance of the Blockchain
blockchain = Blockchain()

# -----
# API Endpoint: Mine a new block
# -----
@app.route('/mine_block', methods=['GET'])
def mine_block():
    # Get the previous block
    previous_block = blockchain.get_previous_block()

```

```

# Extract previous proof
previous_proof = previous_block['proof']

# Run Proof of Work algorithm
proof = blockchain.proof_of_work(previous_proof)

# Get hash of previous block
previous_hash = blockchain.hash(previous_block)

# Create the new block
block = blockchain.create_block(proof, previous_hash)

# Prepare response
response = {
    'message': 'Congratulations, you just mined a block!',
    'index': block['index'],
    'timestamp': block['timestamp'],
    'proof': block['proof'],
    'previous_hash': block['previous_hash']
}
return jsonify(response), 200

# -----
# API Endpoint: Get the full blockchain
# -----
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

```

```
# -----  
# API Endpoint: Check if blockchain is valid  
# -----  
@app.route('/is_valid', methods=['GET'])  
def is_valid():  
    is_valid = blockchain.is_chain_valid(blockchain.chain)  
  
    if is_valid:  
        response = {'message': 'All good. The Blockchain is valid.'}  
    else:  
        response = {'message': 'Houston, we have a problem. The  
Blockchain is not valid.'}  
  
    return jsonify(response), 200  
  
# -----  
# Run the Flask app  
# -----  
app.run(host='0.0.0.0', port=5000)
```

Output:

Pretty-print ☒

```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-24 11:41:29.988365",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "51a955a111f2179a7a16c9a8bcdf89219267bad7967dbc0ecc24958b6d6bf3e7",
      "proof": 533,
      "timestamp": "2026-08-24 11:41:29.991067",
      "transactions": [
        {
          "amount": 10000,
          "receiver": "Gore",
          "sender": "Chinmay"
        },
        {
          "amount": 1000,
          "receiver": "Rakshit",
          "sender": "Gore"
        }
      ]
    }
  ],
  "length": 2
}
```

HTTP

http://127.0.0.1:5000/add_transaction

Save

No Environment

POST

http://127.0.0.1:5000/add_transaction

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

Code

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Beautify

```
1 {
2   ...."sender": "Chinmay",
3   ...."receiver": "HVP",
4   ...."amount": 50000
5 }
```

Body

Cookies

Headers (5)

Test Results

Status: 201 Created Time: 6 ms Size: 300 B

Problems

Output

Debug Console

Terminal

Ports

Postman Console

All Logs Clear

> POST http://127.0.0.1:5000/add_transaction

415 | 6 ms

> POST http://127.0.0.1:5000/add_transaction

201 | 5 ms

> POST http://127.0.0.1:5000/add_transaction

201 | 5 ms

> POST http://127.0.0.1:5000/add_transaction

201 | 4 ms

> POST http://127.0.0.1:5000/add_transaction

201 | 6 ms

> Network

Show raw log

> Request Headers

```
Searching for Golden nonce...

Block mined successfully!

Previous Hash:
860ce8af7da2cd9c41241171f3ccc18cc485c986243782c90c5156e38b96bc3a

Current Hash:
59e1c28e2e6947b9e22eb3d8fde8327a0504da38f0ebe004ddff27d6b6e39137

Complete Block:
{
  "index": 2,
  "timestamp": "2026-08-24 12:01:45.639360",
  "transactions": [
    {
      "sender": "Chinmay",
      "receiver": "Arjun",
      "amount": 100
    },
    {
      "sender": "Rohit",
      "receiver": "Chinmay",

```

```
Modified Transaction:
{'sender': 'Chinmay', 'receiver': 'Arjun', 'amount': 9999}

Stored Current Hash:
59e1c28e2e6947b9e22eb3d8fde8327a0504da38f0ebe004ddff27d6b6e39137

Actual Hash After Modification:
28b42b2f6487a6590014bb7226f41db99d5319f2d3c2ebc99e8f3fdefff099f8
```

Step 14: Validate the Blockchain After Transaction Tampering

```
if blockchain.is_chain_valid(tampered_chain):
    print("RESULT: Tampered Blockchain is VALID.")
else:
    print("RESULT: Tampered Blockchain is NOT VALID.")
    print("Reason: Transaction data was modified, so the stored current hash is no longer correct.")
```

✓ 0.0s

Python

```
RESULT: Tampered Blockchain is NOT VALID.
Reason: Transaction data was modified, so the stored current hash is no longer correct.
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.

```
Administrator: Command Prompt

c:\blockchain-project>python -m venv venv
Unable to copy 'C:\Users\Admin\\AppData\Local\Programs\Python\Python311\venv\launcher.exe'

c:\blockchain-project>pip install Flask==0.12.2
Collecting Flask==0.12.2
  Downloading Flask-0.12.2-py2.py3-none-any.whl.metadata (1.9 kB)
Requirement already satisfied: Jinja2>=2.4 in c:\users\admin\appdata\local\programs\python\python311\lib\site-packages (
  From Flask==0.12.2) (3.1.4)
Collecting Werkzeug>=0.7 (from Flask==0.12.2)
  Downloading werkzeug-3.1.5-py3-none-any.whl.metadata (4.0 kB)
Collecting click>=2.0 (from Flask==0.12.2)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=0.21 (from Flask==0.12.2)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Requirement already satisfied: colorama in c:\users\admin\appdata\roaming\python\python311\site-packages (from click>=2.
  0->Flask==0.12.2) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\admin\appdata\local\programs\python\python311\lib\site-packag
  es (from Jinja2>=2.4->Flask==0.12.2) (2.1.5)
Downloading Flask-0.12.2-py2.py3-none-any.whl (83 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading werkzeug-3.1.5-py3-none-any.whl (225 kB)
Installing collected packages: Werkzeug, itsdangerous, click, Flask
Successfully installed Flask-0.12.2 Werkzeug-3.1.5 click-8.3.1 itsdangerous-2.2.0

[notice] A new release of pip is available: 25.1.1 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip

c:\blockchain-project>
```

```
127.0.0.1:5000/mine_block

Pretty-print ☐

{"index":2,"message":"Congratulations, you just mined a block!","previous_hash":"8c388f32c69f4b417b499e50566d099df5ede943a8d7d58deb98e277fda3a683","proof":533,"timestamp":"2026-01-22 11:01:36.576617"}
```

```
127.0.0.1:5000/get_chain

Pretty-print ☐

{"chain":[{"index":1,"previous_hash":"","proof":1,"timestamp":"2026-01-22 10:59:35.998404"}, {"index":2,"previous_hash":"8c388f32c69f4b417b499e50566d099df5ede943a8d7d58deb98e277fda3a683","proof":533,"timestamp":"2026-01-22 11:01:36.576617"}], "length":2}
```

```
127.0.0.1:5000/is_valid

Pretty-print ☐

{"message":"All good. The Blockchain is valid."}
```